

K-Means in Python with scikit-learn

Contents

1. Overview	2
2. Prerequisites	2
3. The student data	3
4. Choosing k with the elbow method	3
5. Training k-means with $k = 4$	4
6. Plotting the clusters with boolean indexing	5
7. Reading the clusters	6
8. k-means in three dimensions	6
9. Summary	8
10. Key terms	8

Where this fits: Pipeline map steps 8 (Model), 10 (Tune). See the [Course map](#).



- **Builds on:** Unsupervised learning ([Note 3](#)); Feature scaling ([Note 24](#)).
- **Leads to:** Hierarchical clustering ([Note 131](#)); DBSCAN ([Note 132](#)).
- **Compare with:** Hierarchical clustering ([Note 131](#)); DBSCAN ([Note 132](#)).

1. Overview

Key point: With scikit-learn, k-means takes a few lines: plot the elbow curve with `inertia_`, pick `k`, call `fit_predict`, and colour the points by cluster.

This Note runs k-means (the [k-means Note](#)) on 200 students described by CGPA and IQ. The elbow method picks $k = 4$, and k-means finds four groups of students that a placement cell can train in four different ways (Figure 1). Then the same code clusters data with three columns.

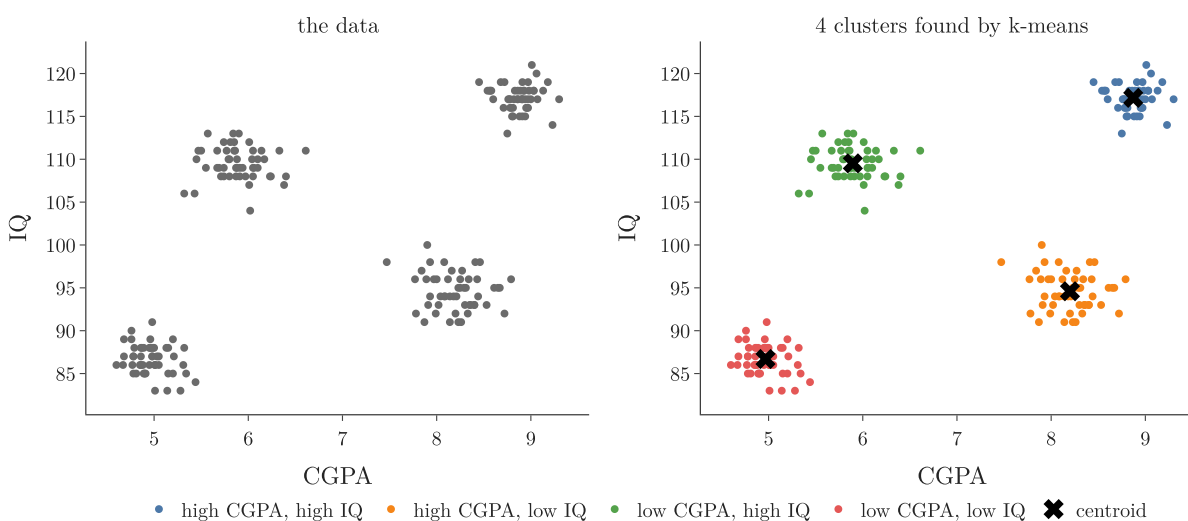


Figure 1: The 200 students (left) and the 4 clusters k-means finds (right); crosses are the centroids

The Notebook (`notebook.ipynb`) runs every step.

2. Prerequisites

- How k-means works and the elbow method: the [k-means Note](#).
- `pandas`, `iloc` and `fit`: the [toy project Note](#).

3. The student data

Key point: 200 final-year students, two columns: `cgpa` and `iq`. No output column: this is unsupervised learning.

The file `data/student_clustering.csv` holds the CGPA and IQ of 200 final-year students. It is a toy dataset, generated for practice, so its groups are much cleaner than in real data.

Python: Loading the data.

```
import pandas as pd

df = pd.read_csv("data/student_clustering.csv")
df.shape      # (200, 2)
df.head()     # columns: cgpa, iq
```

CGPA runs from 4.6 to 9.3 and IQ from 83 to 121. Plotted (Figure 1, left), the students form four clear groups. In 2 dimensions we can count them by eye; the elbow method should agree.

4. Choosing k with the elbow method

Key point: Fit one `KMeans` per `k` from 1 to 10, store each model's `inertia_` (its WCSS), and plot them: the curve flattens at `k = 4`.

`KMeans` lives in `sklearn.cluster`. After fitting, its attribute `inertia_` holds the WCSS of the clusters it found (WCSS: the [k-means Note](#), section 5.1).

Python: The elbow loop.

```
from sklearn.cluster import KMeans

wcss = []
for i in range(1, 11):
    km = KMeans(n_clusters=i, random_state=0)
    km.fit(df)                    # all columns are inputs
    wcss.append(km.inertia_)      # WCSS of this model
n_clusters is k. random_state=0 fixes the random start, so every run gives the same
numbers. Attributes ending in _, like inertia_, exist only after fit.
```

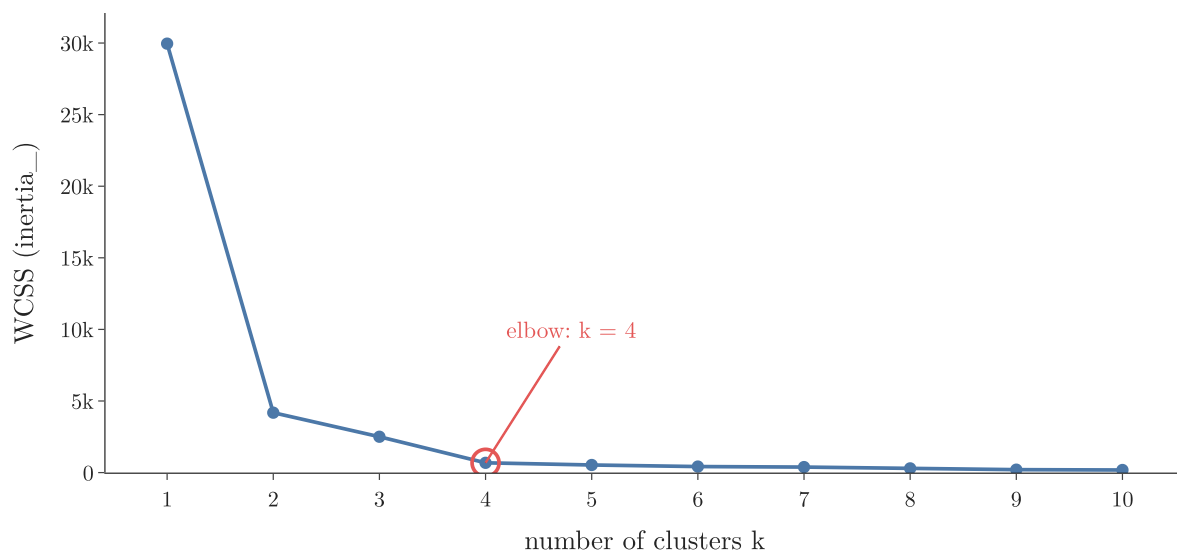


Figure 2: Elbow curve of the 200 students: WCSS falls steeply up to $k = 4$, then stays flat

Figure 2 plots the ten values. WCSS starts at 29,958 for one cluster and drops to 4,184 for two. It keeps falling to 2,503 at $k = 3$ and 682 at $k = 4$. After that it barely moves: 530 at $k = 5$, 421 at $k = 6$.

The curve bends twice: a sharp bend at $k = 2$ and a second one at $k = 4$. After $k = 4$ nothing changes much, so $k = 4$ is the elbow, the same number we counted in Figure 1. Choosing between two bends is a judgement call; plotting the data, when possible, settles it.

5. Training k-means with $k = 4$

Key point: `fit_predict` trains the model and returns one cluster number per row, from 0 to $k - 1$.

Python: Fitting and predicting in one call.

```
X = df.iloc[:, :].values          # the table as a NumPy array
km = KMeans(n_clusters=4, random_state=0)
y_means = km.fit_predict(X)
y_means[:5]                       # array([3, 2, 1, 1, 2])
km.cluster_centers_                # the 4 centroids
```

`.values` turns the DataFrame into a NumPy array, which is easier to slice. `fit_predict` is `fit` followed by returning `labels_`, the cluster of every training row.

The first student is in cluster 3, the second in cluster 2, and so on. The numbers are only names: another run could call the same group 0 instead of 3. Each cluster holds exactly 50 students, and the centroids (`cluster_centers_`) are:

Cluster	Mean CGPA	Mean IQ
0	8.87	117.2
1	8.20	94.6
2	5.89	109.5
3	4.97	86.7

Extra: scikit-learn's `KMeans` does not pick the starting centroids purely at random. Its default `init="k-means++"` picks the first centroid at random from the data. Each next centroid is also a data point, but points far from the centroids chosen so far are more likely to be picked: the chance is proportional to the squared distance. The starting centroids are thus spread out, which avoids most bad starts.

The second default, `n_init="auto"`, sets how many times the whole algorithm restarts from new centroids; the run with the lowest inertia is kept. With `k-means++` it runs once; with `init="random"` it runs 10 times. Other defaults: `n_clusters=8` and `max_iter=300` rounds at most.

Extra: The data was not scaled here, although k-means is distance-based (the [k-means Note](#), section 4.3). Running k-means on standardized columns puts every student in the same group as before: the four groups are so far apart that IQ's larger numbers do not matter. On real data, scale first.

6. Plotting the clusters with boolean indexing

Key point: `X[y_means == 0, 0]` picks the CGPA of every student in cluster 0; one scatter call per cluster colours the groups.

`y_means == 0` is an array of 200 True/False values: **True** where the student is in cluster 0. Using it as a row index keeps only those rows; this is **boolean indexing**. The second index picks the column: 0 for CGPA, 1 for IQ.

Python: One trace per cluster with Plotly.

```
import plotly.graph_objects as go

fig = go.Figure()
for c in range(4):
    fig.add_trace(go.Scatter(
        x=X[y_means == c, 0],    # CGPA of cluster c
        y=X[y_means == c, 1],    # IQ of cluster c
        mode="markers", name=f"cluster {c}"))
fig.show()
```

Figure 1 (right) is the result, with the centroids marked as crosses. Every student landed in the group we would have drawn by hand.

7. Reading the clusters

Key point: Each cluster is a type of student; naming them turns the clusters into a plan.

A data analyst now names each cluster by its centroid (Figure 1, right):

- **High CGPA, high IQ** (blue): intelligent and hard-working. They need little guidance.
- **High CGPA, lower IQ** (orange): hard-working and sincere. They also need little guidance.
- **Lower CGPA, high IQ** (green): smart but relaxed about studies. They need motivation.
- **Low CGPA, low IQ** (red): they need the most training.

The placement cell can now plan four kinds of preparation. The same idea clusters customers, films or images.

8. k-means in three dimensions

Key point: The exact same code clusters data with 3 columns, and would with 100.

To see k-means beyond two columns, we generate 200 points around four centres in three dimensions with `make_blobs`.

Python: Data with three columns.

```
from sklearn.datasets import make_blobs
```

```
centroids = [(-5, -5, 5), (5, 5, -5),  
              (3.5, -2.5, 4), (-2.5, 2.5, -4)]  
X3, _ = make_blobs(n_samples=200, centers=centroids,  
                   cluster_std=1, n_features=3,  
                   random_state=1)
```

`make_blobs` returns the points and the true group of each; `_` is the usual name for a value we ignore. `cluster_std=1` is the spread of each group.

The elbow loop, run for $k = 1$ to 20 without changing a line, gives WCSS 11,144, 4,122, 2,552, 593, then 544: the elbow is again at $k = 4$. Training with $k = 4$ and colouring by cluster gives Figure 3. In the Notebook, `plotly.express.scatter_3d` draws it as a 3-D plot we can rotate.

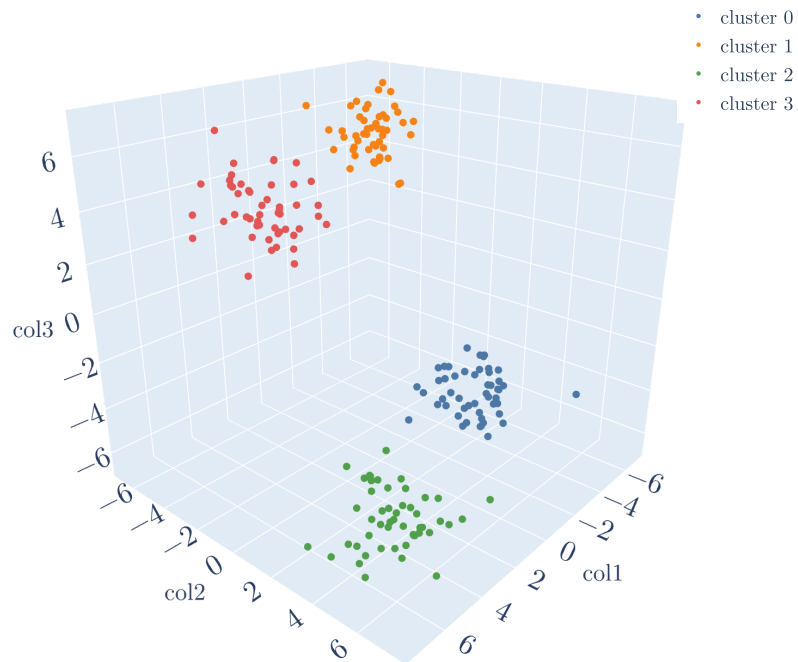


Figure 3: k-means with $k = 4$ on 200 points with three columns

What k-means did in 2 dimensions it does in 3, and in any higher number of dimensions, with the same code. Only the plotting stops at 3.

9. Summary

Task	Code
Load the data	<code>pd.read_csv(...)</code>
WCSS of one model	<code>KMeans(n_clusters=i).fit(df).inertia_</code>
Elbow curve	loop $k = 1$ to 10, plot the WCSS
Train and get clusters	<code>y_means = km.fit_predict(X)</code>
Centroids	<code>km.cluster_centers_</code>
Rows of cluster 0, column 0	<code>X[y_means == 0, 0]</code>
3-D test data	<code>make_blobs(..., n_features=3)</code>

- `inertia_` is scikit-learn's name for WCSS; the elbow on the students is at $k = 4$.
- `fit_predict` returns one cluster number per row; the numbers are labels, not ranks.
- `KMeans` starts with `k-means++` and keeps the best of `n_init` runs.
- Name each cluster by its centroid to turn it into a decision.
- The same code works for any number of columns.

10. Key terms

Term	Meaning
<code>KMeans</code>	scikit-learn's k-means class, in <code>sklearn.cluster</code>
<code>inertia_</code>	The WCSS of a fitted <code>KMeans</code> model
<code>fit_predict</code>	Trains a clustering model and returns the cluster of every row
<code>labels_</code>	The cluster number of every training row, after fitting
<code>cluster_centers_</code>	The coordinates of the final centroids of a fitted <code>KMeans</code>
<code>k-means++</code>	The default start of <code>KMeans</code> : centroids picked one by one, far-away points more likely
<code>n_init</code>	How many times <code>KMeans</code> restarts from new centroids; the run with the lowest inertia is kept
Boolean indexing	Selecting rows with an array of True/False values, e.g. <code>X[y_means == 0]</code>
<code>make_blobs</code>	scikit-learn function that generates points around chosen centres